SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective:

To implement and evaluate a Decision Tree classifier using Python on the Iris dataset.

(a) Dataset Loading, Pre-processing & Splitting

Code:

python

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Display first 5 rows and data types
print(df.head())
print(df.dtypes)

# Check missing values
print(df.isnull().sum())

# Split dataset
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Output Explanation:

- First 5 rows show sepal/petal measurements.
- Data types: all float except target (int).
- No missing values in Iris dataset.

(b) Build Decision Tree (Gini / Information Gain)

Code:

python

```
# Build Decision Tree using Gini Index
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)

# Print tree structure
tree_rules = export_text(model, feature_names=list(X.columns))
print(tree_rules)
```

Root Node Identification:

The root node is typically **petal length (cm)** because:

- It provides the **highest information gain**.
- It best separates Setosa from Versicolor/Virginica.
- It has the **lowest Gini impurity** among all features.

(c) Evaluation: Accuracy, Confusion Matrix, Misclassified Instances

Code:

python

```
# Predictions
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
# Classification Report
print("Classification Report:\n", classification_report(y_test, y_pred))

# Misclassified instances
misclassified = X_test[y_test != y_pred]
print("Misclassified Instances:\n", misclassified)
```

Performance Comments:

- Decision Tree on Iris usually gives **95–100% accuracy**.
- Confusion matrix shows very few misclassifications.
- Example misclassified instances (varies by run):
    - A Virginica classified as Versicolor
    - A Versicolor classified as Virginica

Reason: These two classes overlap in petal dimensions.

Experiment 2: Reinforcement Learning – Q-Learning

Objective:

To implement Q-Learning in a 4×4 Grid World and observe the learned optimal policy.

(a) Environment Definition + Q-table Initialization

Grid World Description:

- **States:** 16 (0–15)
- **Actions:** Up, Down, Left, Right
- **Rewards:**
    - Goal state: **+10**
    - Obstacle: **−5**
    - Each step: **−1**

Hyperparameters:

- Learning rate $\alpha = 0.7$

- Discount factor $\gamma = 0.9$
- Exploration $\varepsilon = 0.2$

Code:

python

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Grid World (4x4)
n_states = 16
n_actions = 4  # Up, Down, Left, Right

# Rewards
goal_state = 15
obstacle_state = 5

# Q-table
Q = np.zeros((n_states, n_actions))

# Hyperparameters
alpha = 0.7
gamma = 0.9
epsilon = 0.2

# Action mapping
def get_next_state(state, action):
    row = state // 4
    col = state % 4

    if action == 0 and row > 0:      # Up
        row -= 1
    elif action == 1 and row < 3:    # Down
        row += 1
    elif action == 2 and col > 0:    # Left
```

```
        col -= 1
    elif action == 3 and col < 3:    # Right
        col += 1

    return row * 4 + col

def get_reward(state):
    if state == goal_state:
        return 10
    elif state == obstacle_state:
        return -5
    else:
        return -1

# Training
episodes = 100
rewards_per_episode = []

for ep in range(episodes):
    state = 0
    total_reward = 0

    while state != goal_state:
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state])

        next_state = get_next_state(state, action)
        reward = get_reward(next_state)

        Q[state, action] = Q[state, action] + alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )
```

```
        state = next_state
        total_reward += reward

    rewards_per_episode.append(total_reward)

    # Print Q-values at episodes 1, 50, 100
    if ep in [0, 49, 99]:
        print(f"\nQ-values at Episode {ep+1}:")
        print(Q[0])   # State 0
        print(Q[10])  # State 10 (representative)
```

(b) Q-table Evolution

Representative Output (Example):

Episode	Q(State 0) Values	Q(State 10) Values
1	[0, -1, 0, -1]	[-1, -1, -1, -1]
50	[3.2, 4.1, 2.8, 4.5]	[5.0, 6.2, 4.9, 6.5]
100	[6.8, 7.1, 6.5, 7.4]	[8.5, 9.1, 8.2, 9.4]

Q-values increase as the agent learns better paths.

(c) Final Learned Policy + Convergence Plot

Extract Policy:

```
python
policy = np.argmax(Q, axis=1)
print("Final Policy (0=Up,1=Down,2=Left,3=Right):")
print(policy.reshape(4,4))
```

Example Learned Policy Grid:

State	Best Action
0	Right
1	Right
2	Down
3	Down
...	...
15	Goal

Plot Cumulative Reward:

python

```
plt.plot(rewards_per_episode)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Convergence")
plt.show()
```

Convergence Justification:

- Rewards increase over episodes.
- Agent gradually reduces unnecessary moves.
- Q-values stabilize → policy converges.
- Exploration (ε) ensures the agent avoids local optima.